

TP16 – SYSTÈME DE GESTION D'ADHÉRENTS

1. INTRODUCTION

Galette (<https://galette.eu>) est un gestionnaire d'adhérents à destination d'associations. Il gère les informations personnelles des adhérents, l'historique des cotisations, etc.

Il stocke ses données dans une base de données SQL. Le schéma complet de la base est présent dans les sources du logiciel (<https://github.com/galette/galette/tree/develop/galette/install/scripts>). Pour ce sujet, je vous donne une version simplifiée, lisible dans le fichier `schema-pg.sql`.

Ce fichier contient de nombreux commentaires qui permettent de comprendre la structure des données.

Votre travail consiste à proposer des requêtes SQL pour des tâches plus ou moins usuelles. Certaines correspondent à des requêtes que j'ai dû écrire pour développer un greffon spécifique (et certaines correspondent à des requêtes que je dois écrire pour une nouvelle version du greffon).

2. CLIENT POSTGRESQL

On joue dans le terminal. La base de données est installée sur la machine `c009-01`. En ligne de commande, le client `psql` vous connecte au serveur PostgreSQL de cette machine avec la ligne suivante :

```
psql -h c009-01 mpigalette mpi
```

SHELL

Ceci vous donne accès à la base de données `mpigalette` avec le nom d'utilisateur `mpi` dont le mot de passe est `mpi`. Vous n'avez (normalement) qu'un accès en lecture seule à cette base de données.

3. EXPLORATION DES ADHÉRENTS

Les adhérents ont un identifiant de statut (selon leur fonction dans l'association). Les membres actifs ont le statut 10. Les membres du bureau ont un statut entre 0 et 5.

Question 1. Déterminer les membres du bureau. Déterminer le nombre de membres pour chaque statut.

Les adhérents sont membres de groupes. Les groupes possèdent des responsables.

Question 2. Qui sont les responsables de groupes ?

Question 3. À quels groupes manque-t-il un responsable ?

Question 4. Étant donné un adhérent, les personnes suivantes peuvent mettre à jour sa fiche : lui-

même, les membres du bureau, les responsables du ou des groupes dont est membre l'adhérent. Déterminer, étant donné un identifiant d'adhérent si un autre identifiant donné peut modifier sa fiche.

Question 5. On veut contacter, pour chaque groupe, soit son responsable, soit l'ensemble des adhérents du groupe s'il n'y a pas de responsable. Avec quelle requête faites-vous cela ?

4. GESTION DES COTISATIONS

Question 6. Quels sont les adhérents à jour de cotisation ? Combien sont-ils ? On fera attention au fait que seuls les types de cotisations dont le champ `cotis_extension` est vrai donnent lieu à une extension d'adhésion. Les autres types correspondent à de simples dons à l'association.

Question 7. Avez-vous remarqué la présence du booléen `bool_exempt_adh` qui indique qu'un adhérent est dispensé de cotisation, donc est toujours considéré à jour de cotisation ?

Question 8. Déterminer la liste des adhérents à qui envoyer un rappel de cotisation car il sont à moins de 10 jours de l'échéance.

Question 9. Une même personne peut faire un paiement global, qui s'appelle une transaction. Cette transaction regroupe le paiement d'une ou plusieurs cotisations, pour un ou plusieurs adhérents. Le montant total de la transaction est stocké dans le champ `trans_amount`. Vérifier que la somme des cotisations associées à une transaction donnée ne dépasse pas le montant de la transaction. Pour quelles transactions ne correspond-il pas exactement ?

Question 10. Pour l'édition d'un reçu fiscal annuel, on souhaite obtenir pour chaque adhérent le total des sommes versées entre le 1er janvier et le 31 décembre (inclus) de l'année précédente. Le reçu fiscal demande de préciser une date de paiement : on retient pour cela la date du dernier paiement.

5. LISTES DE DIFFUSION

Les adhérents à jour de cotisation sont inscrits automatiquement sur des listes de diffusion. Ces listes sont énumérées dans la table `galette_ml_lists`. La liste `officielle` contient tous les adhérents à jour de cotisation. La liste `bureau` contient tous les membres du bureau, c'est-à-dire par convention ceux dont le numéro de statut est inférieur à 5, quel que soit leur statut d'adhésion. Les adhérents concernés sont automatiquement inscrits ou désinscrits selon ces critères.

Pour les autres listes, l'inscription peut être contrôlée par les adhérents :

- certaines listes sont accessibles à tout le monde (`blagues` , `generale`),
- certaines listes ne sont accessibles qu'à certains profils d'adhérents,
- pour chaque liste et pour chaque profil, on indique dans la table

`galette_ml_lists_profiles` si un adhérent qui a ce profil est inscrit par défaut ou non, s'il peut voir la liste dans l'interface et s'il a le droit de changer son statut d'inscription.

Quand l'adhérent a le droit de changer son statut d'inscription pour une liste, on renseigne son choix dans la table `galette_ml_lists_subscriptions` : l'inscription est alors contrôlée par le booléen `is_subscribed` pour un adhérent et une liste donnée ; s'il n'y a pas de ligne correspondante dans cette table, on adopte le comportement par défaut pour le profil et la liste.

Toutes les requêtes de cette partie doivent renvoyer une seule colonne, contenant les adresses e-mail des adhérents concernés par la requête, sans doublons.

Question 11. Écrire une requête qui liste tous les membres du bureau, pour leur inscription sur bureau .

Question 12. Écrire une requête qui liste tous les adhérents à jour de cotisation, pour leur inscription sur officielle .

Question 13. En réalité, on veut accorder un petit délai aux adhérents dont la cotisation a expiré. Ils ont le droit à 45 jours après leur date d'échéance de cotisation avant d'être retirés de la liste officielle . Adapter la requête précédente.

Question 14. Étant donné une liste à inscription variable, déterminer l'ensemble des adhérents inscrits à cette liste.

6. DEPUIS UN PROGRAMME

Les gestionnaires de bases de données proposent tous des fonctions, utilisables par exemple depuis OCaml ou C, pour exécuter des requêtes et obtenir les résultats. Dans le cas de PostgreSQL, ces fonctions sont fournies par la `libpq` dont la documentation est en ligne : <https://www.postgresql.org/docs/current/libpq.html>

En OCaml, la `libpq` a des bindings par exemple avec PostgreSQL-OCaml : <https://mmottl.github.io/postgresql-ocaml/>

Question 15. Exécutez l'une des requêtes précédentes depuis vos langages de programmation préférés. Comment créer une fonction C ou OCaml qui prend en paramètre un entier `id_adh` et qui effectue une requête sur cet identifiant, sans risquer de créer une faille de sécurité majeure ?